Fine-Tuning Self-Supervised Speech Representations for Hindi–English Code-Switched Speech Recognition: A Controlled Study of Data Quality, Script Normalisation and Language-Model Decoding

Nehal Abbott (2024AIB1010) Shlok (2024AIB1016)
Indian Institute of Technology Ropar, Rupnagar, Punjab, India

***Abstract*—Hindi–English code-switched speech combines two phonologies within an utterance, writes English words in either Devanagari or Latin script, and offers little labelled data, which together make automatic speech recognition (ASR) difficult. This paper documents a compute-constrained recognition system for the MUCS 2021 Subtask-2 corpus, about 90 hours of code-switched spoken tutorials. A wav2vec 2.0 base encoder of the Vakyansh Hindi lineage is fine-tuned with connectionist temporal classification (CTC) over a 120-symbol mixed-script character inventory, and decoded either greedily or by beam search guided by a 5-gram Kneser–Ney language model trained only on training transcripts. Five iterations, each changing one factor, examine continued training, script normalisation of the training targets, removal of misaligned training utterances, and CTC-based realignment of segment boundaries. Every model and decoder choice was made on a held-out validation set with a script-folded word error rate (WER); test and blind-test sets were decoded once per model and never used for selection. The selected system (iteration 4) reaches a blind-test WER of 20.44% and a transliterated WER (T-WER) of 19.20%, both confirmed with the organisers' scoring script, against 25.53% and 23.80% for the official challenge baseline. Error analysis attributes most remaining errors to English technical vocabulary, segment-boundary effects and inconsistent script use, and shows that realignment improves recognition of correctly segmented speech while lowering scores under the released segmentation. All results come from single training runs.**

***Index Terms*—code-switching, automatic speech recognition, wav2vec 2.0, connectionist temporal classification, n-gram language model, Hindi–English**

I. Introduction

Automatic speech recognition (ASR) has moved from hybrid hidden Markov model systems towards end-to-end neural models, and more recently towards encoders pretrained by self-supervision on untranscribed audio and fine-tuned on modest amounts of labelled speech [1], [2]. These advances have been measured mostly on monolingual benchmarks. Multilingual speakers, however, frequently alternate between languages within a single utterance. In India, Hindi–English code-switching is common in education, technology and media, where English technical terms are embedded in Hindi sentences [3].

Code-switched speech is harder to recognise than either language alone for three reasons. First, the acoustic model must handle two phoneme inventories and the transitions between them inside one utterance. Second, labelled code-switched speech is scarce: the largest openly licensed Hindi–English corpus, MUCS 2021 Subtask-2, contains about 90 hours of training audio [3], [4]. Third, the written form is not standardised. English words appear in Latin script in some transcripts and in Devanagari transliteration in others, so a hypothesis can be phonetically correct and still count as a word error [5]. The MUCS organisers report that more than half of all errors fall on English words, which make up about a third of the reference words, and introduce a transliterated WER (T-WER) to reduce the penalty for script choice [3].

Chadha et al. [6] showed that a wav2vec 2.0 encoder pretrained on Hindi audio, fine-tuned with connectionist temporal classification (CTC) [7] and decoded with an n-gram language model (LM) [8], reaches 21.77% WER on the MUCS blind test without the lexicon engineering of hybrid systems. Their report omits most training hyperparameters, however, and published MUCS systems rarely isolate the effect of individual data-quality interventions or report how decisions were made [6], [9].

Problem statement. This work asks how far a single-GPU, self-supervised CTC system can be improved on Hindi–English code-switched speech by interventions on training duration, target script and training-segment quality, when every decision is made on validation data only and the effect of each intervention is measured in isolation.

Contributions. The paper makes four contributions:

- A documented implementation of wav2vec 2.0 fine-tuning with CTC and 5-gram KenLM beam search on MUCS 2021 Subtask-2, with strict separation of training, validation, test and blind-test data. Blind-test WER and T-WER are confirmed with the organisers' scoring script.
- A controlled study of five iterations, each changing one factor: continued training, script-normalised targets, a misalignment filter and CTC realignment of training segments. Iterations are selected with a script-folded validation WER that is insensitive to the script in which English words are written.
- Evidence that data-quality interventions interact with the conventions of the evaluation data: script normalisation and realignment lower blind-test error but not validation error, and an analysis attributes the disagreement to how each split writes English words and transcribes speech at segment boundaries.
- An error analysis by error type, script, language-switch position and utterance duration, and a CPU-compatible inference path whose pure-Python ARPA language-model reader was validated against KenLM.

Section II reviews related work, Section III describes the data, Section IV the method and Section V the experimental design. Sections VI and VII report results and errors, Section VIII the implementation, and Sections IX–XI discuss findings, limitations and conclusions. Fig. 1 summarises the system studied: a self-supervised wav2vec 2.0 encoder with

a CTC output layer over a mixed-script character inventory, decoded greedily or with an n-gram LM.

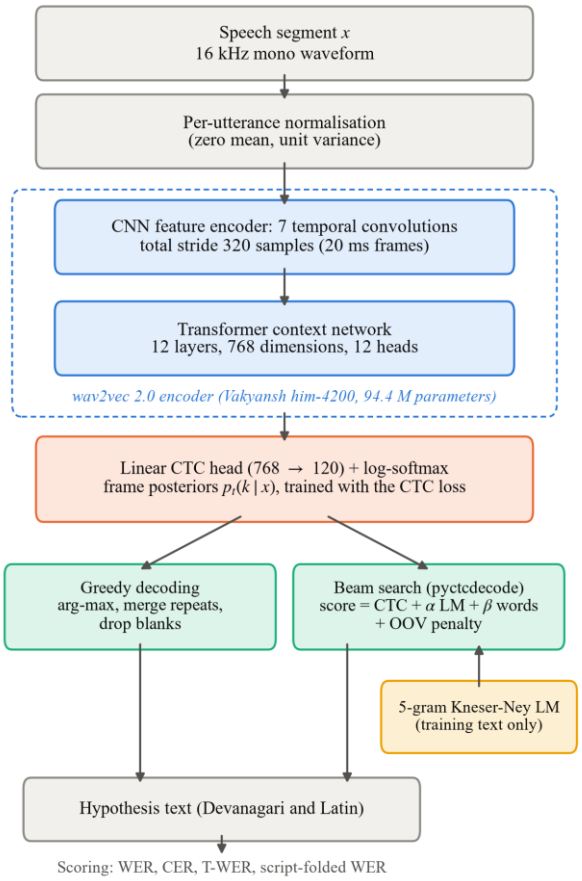


Fig. 1. System architecture. The encoder and CTC head are fine-tuned jointly; the language model is used only at decoding time. Script-folded WER is used for model selection on validation data, and T-WER is the official blind-test metric.

II. Related Work

A. Self-supervised and multilingual speech representations

wav2vec 2.0 learns contextual speech representations by masking latent features of raw audio and solving a contrastive task over quantised targets; fine-tuning with a CTC head on a few hours of labelled speech yields competitive recognition [1]. XLSR extends the approach to cross-lingual pretraining on 53 languages [2]. For Indian languages, CLSRIL-23 pretrains a wav2vec 2.0 model on about 10,000 hours of audio in 23 Indic languages [10], and the Vakyansh toolkit releases pretrained and fine-tuned Indic checkpoints together with data-preparation tools [11]. Weakly supervised encoder–decoder models such as Whisper offer an alternative foundation [12].

B. Code-switched speech recognition

Early Hindi–English work relied on proprietary corpora and hybrid systems. Taneja et al. synthesised code-mixed speech by splicing monolingual fragments [13], and Sharma et al. augmented training with code-switched text-to-speech [14]. Srivastava et al. found end-to-end CTC and attention models 15–30 WER points worse than hybrid TDNN systems on 47 hours of conversational data [15], suggesting that from-scratch end-to-end models need more data than such corpora provide. Later work studied transformer transducers with monolingual pretraining [16], concatenated tokenisers that emit language identity [17], and Whisper adaptation with synthetic code-mixed data [18]. These studies use different data and metrics, which limits direct comparison.

C. The MUCS 2021 challenge

MUCS 2021 released Hindi–English and Bengali–English code-switched corpora with GMM-HMM, TDNN and transformer baselines, and defined T-WER [3]. The organisers noted that the released segment timestamps are imprecise and reported baselines trained on unaligned and realigned segments. The strongest Hindi–English blind-test WER, 16.43%, came from a hybrid LF-MMI system with a learned lexicon, transliteration maps and removal of badly aligned segments [9]. The best T-WER, 15.64%, came from a CNN-TDNN system with phoneme mapping for English words and external data [19]. A dual-script end-to-end model reached 22.06% [20], [21], and the wav2vec 2.0 system of Chadha et al. 21.77% [6]. Verma et al. addressed script inconsistency by transliterating all text into a common representation before fine-tuning an Indic wav2vec 2.0 model [22].

D. CTC and language-model decoding

CTC trains a frame-level classifier against an unsegmented label sequence by summing over all alignments [7]. Its frame-independent outputs are commonly combined with an n-gram LM in a prefix beam search [23]. Kneser–Ney smoothing remains the standard for such LMs [24], [25], and KenLM provides efficient querying [8]. For code-switching specifically, dual language models that combine monolingual LMs through a switching model have been proposed [26].

E. Research gap

Among MUCS Hindi–English systems, the self-supervised CTC approach is the most reproducible on a single GPU, but its published description lacks training details and ablations [6]. The literature reviewed for this project reports neither random seeds nor run-to-run variance, and does not separate the effect of training-data quality from that of evaluation conventions such as script choice and segment boundaries. The present work addresses this gap with a controlled, validation-only study of one system, and reports every split, including those on which an intervention does not help.

III. Dataset and Preprocessing

A. MUCS 2021 Subtask-2 Hindi–English corpus

The corpus consists of recordings of technical spoken tutorials in which Hindi sentences embed English terms, released under CC BY-SA 4.0 [3], [4]. It is lecture-style speech, not conversational speech. Training audio totals 89.86 hours of whole recordings. A full audit found all 586 recordings to be 16 kHz, mono, 16-bit PCM, with no missing, corrupt, silent or non-finite files. Table I summarises the segments used. A validation set of 4.01 hours was held out from the training split as 22 whole recordings, selected with a fixed seed, so that no recording contributes to both training and validation. The released test and blind-test sets were kept unchanged.

TABLE I
SEGMENT STATISTICS OF THE MUCS 2021 HINDI–ENGLISH DATA AS USED

Split	Rec.	Utt.	Hours	Latin words	Sent. in train
Train	499	50,509	85.55	25.11%	–
Val (from train)	22	2,316	4.01	23.76%	13.21%
Test	30	3,136	5.18	24.26%	41.55%
Blind	35	4,034	6.24	32.27%	4.55%

Rec.: recordings; Utt.: segments. Latin words: share of words in Latin script. Sent. in train: share of utterances whose cleaned sentence also occurs in the training split (train minus val). Blind contains 8 empty references, which are skipped in scoring.

Three properties of the corpus shape the experiments. First, blind references write English in Latin script more often than the other splits (32.27% against 25.11% for training), reflecting a different annotation convention. Second, 41.55% of test utterances repeat a training sentence against 4.55% for blind, so test results may partly reflect memorisation. Third, segment boundaries are coarse: 99% of segments start and end on a whole second in every split, and some recordings carry transcripts offset by several seconds from the audio [3].

B. Audio preprocessing

Fig. 2 shows the preprocessing pipeline. Segments are read from the recordings by their time stamps, down-mixed to mono by channel averaging and resampled to 16 kHz with a polyphase filter. For MUCS audio the last two steps are identities, since every recording is already 16 kHz mono, but they let the inference path accept arbitrary audio. Each segment is normalised to zero mean and unit variance by the encoder's feature extractor, as in pretraining, so no separate loudness normalisation is applied. No audio was truncated. Training excludes segments shorter than 0.5 s or longer than 60 s, which removes none, and 7 segments whose transcripts contain more characters than the encoder produces frames, which CTC cannot align. Training therefore used 50,502 segments.

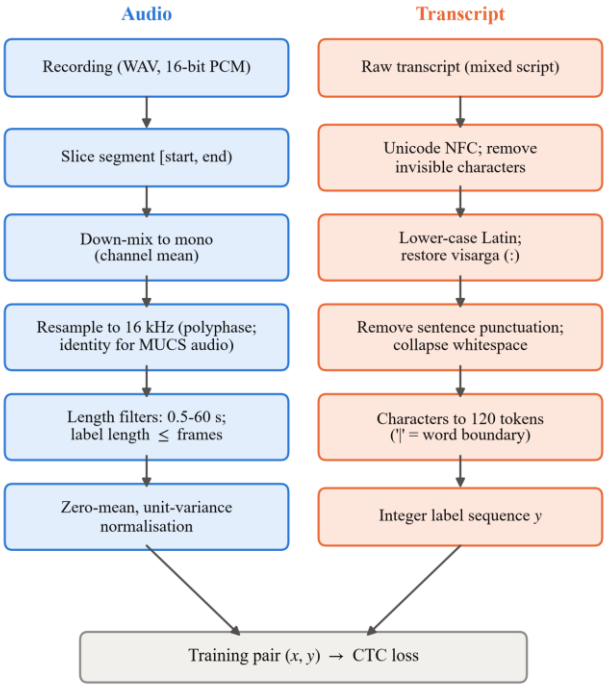


Fig. 2. Audio and transcript preprocessing. Resampling and down-mixing are identities for the 16 kHz mono MUCS recordings but are applied to arbitrary input at inference time.

C. Text normalisation and tokenizer

Transcripts are normalised to Unicode NFC, stripped of invisible formatting characters, lower-cased (affecting Latin script only), cleared of sentence punctuation and collapsed to single spaces. Twelve word stems in which an ASCII colon stands for the Devanagari visarga are restored to the proper character. Scripts are preserved: a word stays in the script chosen by the transcriber, and the baseline applies no transliteration.

The tokenizer is a character inventory built from training transcripts only. It contains 120 symbols: 63 Devanagari characters, 26 lower-case Latin letters, 10 digits, 16 symbols that are spoken in programming tutorials (for example `*` and `+`), and 5 special tokens. The CTC blank is `<pad>` (index 0) and the word delimiter is `|`. Characters occurring fewer than 10 times are removed, following Chadha et al. [6]. Encoding followed by decoding reproduces 99.915% of training transcripts exactly; the remainder differ only in removed rare characters.

IV. PROPOSED METHODOLOGY

The system of Fig. 1 has three parts: a wav2vec 2.0 encoder, a linear CTC output layer, and two decoders, one greedy and one guided by an n-gram LM.

A. Acoustic encoder

The encoder is `Harveenchadha/vakyansh-wav2vec2-hindi-him-4200`, a wav2vec 2.0 base model with 94.42 M parameters [1], [11]. A seven-layer convolutional feature encoder (kernels 10, 3, 3, 3, 3, 2, 2; strides 5, 2, 2, 2, 2, 2, 2) maps 16 kHz audio to 512-dimensional frames at a 20 ms hop (stride 320 samples). A twelve-layer transformer with 768 hidden units and 12 attention heads then produces contextual representations $\mathbf{h}_t$. The checkpoint belongs to the same Vakyansh Hindi lineage as the encoder of Chadha et al. [6], but whether its weights are identical to theirs could not be verified: that paper describes its encoder as pretrained on unlabelled Hindi audio, whereas this checkpoint's model card describes fine-tuning on 4,200 hours of labelled Hindi from a CLSRIL-23 initialisation [10]. The study therefore reproduces the method, not the published number.

B. CTC output layer

A linear layer maps each 768-dimensional frame representation to 120 logits, followed by a log-softmax, giving frame posteriors $p_t(k \mid \mathbf{x})$ over the inventory. Of the 120 rows, 62 are copied from the checkpoint's output layer for characters it already modelled, and 58 are newly initialised, including all Latin letters and digits. Testing showed that the checkpoint's actual blank symbol is `<s>`, which occupies 77.2% of frames on real speech, rather than the padding token named in its configuration. This blank is mapped to the new blank index; trusting the configuration produced empty output.

For a target label sequence $\mathbf{y}$ and input $\mathbf{x}$ with T frames, CTC [7] marginalises over all frame-level paths π that collapse to $\mathbf{y}$ under the mapping $\mathcal{B}$, which merges repeated labels and removes blanks:

$$P(y|x) = \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^{T} p_t(\pi_t|x) \quad (1)$$

and the training loss is the negative log-likelihood

$$\mathcal{L}_{\mathrm{CTC}} = -\ln P(y|x) \quad (2)$$

averaged over the utterances of a batch. CTC needs no frame-level alignment, only that the label sequence is no longer than the frame sequence, which motivates the length filter of Section III.

C. Greedy decoding

Greedy decoding selects the most probable symbol in each frame and applies $\mathcal{B}$:

$$\hat{y} = \mathcal{B}(k_1, \dots, k_T),\ k_t = \operatorname{argmax}_k p_t(k|x) \qquad (3)$$

followed by mapping the delimiter to a space and removing the special tokens that are never training targets. Greedy decoding needs no language model and isolates the acoustic model in the comparisons below.

D. Language model and beam search

A 5-gram word LM was trained on the 50,509 training transcripts (604,999 tokens, vocabulary 17,005 words) after the same normalisation as the acoustic targets. Validation text was excluded because it is used for tuning, and test and blind-test text was never read. The model uses interpolated Kneser–Ney smoothing [24], [25]:

$$\mathrm{P_{KN}}(w|h) = \frac{\max(c(hw) - D_n, 0)}{c(h)} + \lambda(h)\mathrm{P_{KN}}(w|h') \qquad (4)$$

where h is the history, h' its shortened form, D_n an order-specific discount (0.61, 0.74, 0.85, 0.90 and 0.82 for orders 1–5), $\lambda(h)$ the normalising back-off weight, and lower orders use continuation counts. All observed n-grams were kept (17,008 / 157,702 / 304,587 / 368,621 / 378,722 by order). The model is stored in the ARPA text format (68.5 MB) and compiled to the KenLM binary format (27.45 MB) [8]. Because `lmplz` could not be built on the development machine, the ARPA file was produced by a Kneser–Ney builder implemented in the project and compiled with KenLM's `build_binary`.

Beam search uses pyctcdecode [27], which extends CTC prefix beam search [23] with word-level LM scores. A word sequence **w** is scored as

$$Q(w) = \ln \mathrm{P_{CTC}} + \alpha \ln \mathrm{P_{LM}} + \beta|w| + \gamma N_{\mathrm{OOV}} \qquad (5)$$

where $\mathrm{P_{CTC}} = P(\mathbf{w} \mid \mathbf{x})$ is the acoustic probability of the word sequence, $\mathrm{P_{LM}} = P(\mathbf{w})$ its LM probability, $|\mathbf{w}|$ its number of words, N_{OOV} the number of its words outside the LM vocabulary, α the LM weight, β a word-insertion score, and $\gamma \le 0$ a penalty for each word outside the LM vocabulary. These and the beam width are tuned on validation data (Section V). Out-of-vocabulary rates of the LM are 2.69%, 2.95% and 3.65% of word tokens on validation, test and blind, rising to 8.47% of blind's Latin-script tokens.

E. Evaluation metrics

WER is the word-level edit distance between reference and hypothesis, summed over utterances:

$$\mathrm{WER} = \frac{S+D+I}{N} \times 100\% \qquad (6)$$

where S, D and I are substitutions, deletions and insertions, and N is the number of reference words. Scoring follows the organisers' rules: both sides are lower-cased, spoken punctuation in the hypothesis is mapped to symbols, and empty references are skipped. Character error rate (CER) applies the same formula to Unicode code points, including spaces; it is not an official challenge metric. T-WER additionally maps each hypothesis word through the organisers' word list μ, which converts Devanagari spellings of English words to Latin:

$$\text{T-WER} = \frac{\sum_u \mathrm{ED}(r_u, \mu(h_u))}{\sum_u |r_u|} \times 100\% \qquad (7)$$

where ED is the word edit distance and r_u, h_u are the reference and hypothesis of utterance u. T-WER is official only on the blind test. On the test set its references are not annotated for it, so a two-sided approximation is reported and labelled as such. Because the word list was built from blind references, it is used for scoring only and never for training or tuning. For model selection the study defines a script-folded WER,

$$\mathrm{WER_{fold}} = \frac{\sum_u \mathrm{ED}(\varphi(r_u), \varphi(h_u))}{\sum_u |r_u|} \times 100\% \qquad (8)$$

in which φ maps both reference and hypothesis through a training-only map of 587 English words from Devanagari to Latin script. The map is built from training transcripts and the CMU Pronouncing Dictionary [28]: a Devanagari word is mapped when its pronunciation, rendered by rule from the dictionary, matches a Latin training word and the two occur in similar contexts. Script-folded WER counts a word as correct in either script, so iterations that change the script of their targets are compared fairly.

V. Experimental Design

A. Protocol and selection

All experiments follow the protocol of Algorithm 1. Each iteration changes exactly one factor relative to the iteration it builds on. The decoder is tuned on validation data in five stages: a 7 × 5 grid of α and β at beam width 50, 14 finer points around the best, three OOV penalties, beam widths from 50 to 3,000 (a wider beam had to improve WER by at least 0.05 points to be chosen), and a check of milder OOV penalties. Each iteration evaluated 58–61 configurations. The chosen settings were committed to version control before test or blind-test audio was decoded. Iterations are compared by

$$i^* = \operatorname{argmin}_i \mathrm{WER}_{\mathrm{fold}}^{\mathrm{val}}(i), \qquad (9)$$

the beam-search, script-folded validation WER of Eq. (8). Test and blind-test results are reported for every iteration but played no part in any choice.

Algorithm 1. Protocol of one iteration i

Input: training segments $D_{\mathrm{tr}}^{(i)}$, validation set D_{val}, initial weights θ_0, seed 42.
1: Fine-tune θ on $D_{\mathrm{tr}}^{(i)}$ with the CTC loss of Eq. (2); every 2,000 updates, keep the checkpoint with the lowest greedy validation WER.
2: Cache frame log-posteriors for validation, test and blind (one forward pass per utterance, float16).
3: Tune (α, β, γ, beam width) of Eq. (5) on D_{val} only; freeze and commit the decoder configuration.
4: Decode test and blind once with the frozen configuration; score WER, CER and T-WER; confirm blind scores with the organisers' script.
5: Record $\mathrm{WER}_{\mathrm{fold}}^{\mathrm{val}}(i)$; select the final model with Eq. (9).
Output: checkpoint, frozen decoder configuration, and metrics for all splits.

B. Baseline configuration (iteration 1)

Table II lists the training configuration of the baseline. The whole network, including the convolutional feature encoder, was fine-tuned. Three parameter groups were used because a pilot comparison on 150 validation utterances showed that a uniform learning rate of 10^{-5} never produced Latin-script output (0% of output words after 4,000 updates), whereas raising the transformer and head to 10^{-4}

produced 29.6% Latin words after 2,000 updates, with validation loss 1.17 against 1.58. A faster head alone did not help, which indicates that the transformer, not the output layer, must learn when English is being spoken. The convolutional encoder kept 10^{-5} because a single update at 10^{-4} sharply increased error in a smoke test.

TABLE II
BASELINE TRAINING CONFIGURATION (ITERATION 1)

Setting	Value
Encoder	him-4200, wav2vec 2.0 base, 94.42 M parameters, fully fine-tuned
Updates / batch	38,000 updates (about 6 epochs); 1 utterance per step, 8-step accumulation
Learning rates	CNN 1×10^{-5}; transformer and CTC head 1×10^{-4}
Schedule	Tri-stage: 10% linear warm-up, 40% hold, 50% cosine decay to zero
Optimiser	AdamW [29], $\beta = (0.9, 0.98)$, weight decay 0.01, gradient clipping 1.0
Regularisation	Time masking $p = 0.065$, span 10 frames [1], [30]
Precision / memory	FP16 autocast with gradient scaling; gradient checkpointing
Selection	Greedy validation WER every 2,000 updates; best at update 37,872
Seed / hardware	42 (single run); RTX 3050 6 GB laptop GPU; 8 h 52 min

C. Iterations

Table III summarises the five iterations and Fig. 3 their chronology. Iterations 2–5 start from the best checkpoint of iteration 1 with a fresh optimiser and schedule.

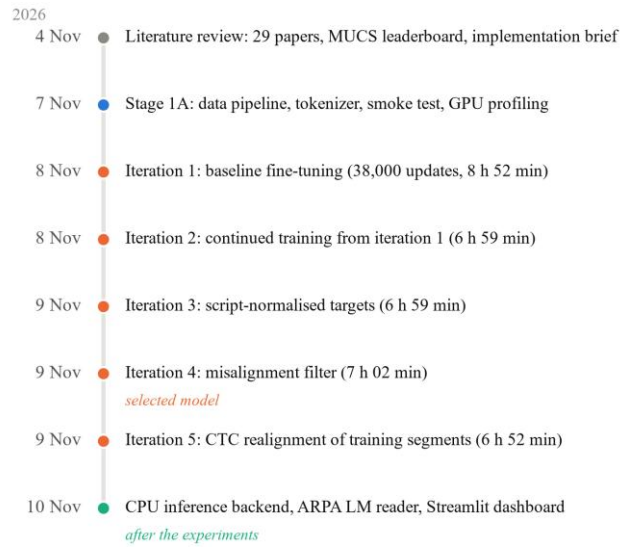


Fig. 3. Research timeline (2026). Dates are those of the research documents, tagged iterations and implementation commits. Durations are training times on the RTX 3050. The inference backend and dashboard were developed after all experiments.

TABLE III
EXPERIMENTAL ITERATIONS

It.	Change (builds on)	Train data	Decoder (α, β, γ, beam)
1	Baseline fine-tuning from him-4200	50,502 utt.	0.35, 1.0, −5, 500
2	Continued training, 44,000 updates, halved learning rates, duration batching (1)	50,502 utt.	0.40, 1.0, −5, 500
3	As 2, English words in Devanagari mapped to Latin in targets and LM (1)	50,502 utt.	0.40, 0.5, −5, 1000
4	As 2, misaligned training utterances removed (2)	46,959 utt., 79.90 h	0.50, 0.0, −10, 200
5	As 4, training segments re-cut by CTC alignment instead of removed (4)	48,805 seg., 81.09 h	0.35, 1.0, −5, 50

Iteration 2 (continued training). Iteration 1's validation WER was still falling when its schedule ended. Iteration 2 continued for 44,000 updates at half the peak learning rates (5×10^{-6} for the CNN, 5×10^{-5} elsewhere) with a 5% warm-up and 35% hold. Batches were grouped by duration up to 60 s of audio per step without accumulation, which was measured to be 1.74× faster than one utterance per step. Training took 6 h 59 min.

Iteration 3 (script normalisation). The baseline error analysis found that words in the wrong script cost about 1.3 WER points on blind. Iteration 3 used iteration 2's recipe with targets in which the 587 mapped English words (3.67% of training tokens) are written in Latin script; the LM was rebuilt on the mapped text. It was compared with iteration 2 to measure the effect of the targets alone.

Iteration 4 (misalignment filter). Because segment boundaries are coarse, some training transcripts describe speech that is not in their audio, which teaches CTC to invent or skip words. Each training utterance was decoded greedily with iteration 1's model and removed when

$$\min(\mathrm{CER}_{\mathrm{raw}}, \mathrm{CER}_{\mathrm{skel}}) > 50, \tag{10}$$

where $\mathrm{CER}_{\mathrm{raw}}$ compares the strings as written and $\mathrm{CER}_{\mathrm{skel}}$ compares consonant skeletons after spelling English words in Devanagari via the pronouncing dictionary, so that utterances differing only in script are kept. The threshold was set by inspecting random training utterances before any model was trained on the result: all 12 inspected above 60 were misaligned, as were 26 of 28 between 50 and 60, while the 40–50 band mixed misalignment with ordinary recognition errors. The filter removed 3,550 utterances (7.03%, 5.65 h); the commonest case was a full sentence paired with 2–3 s of audio. Otherwise iteration 4 repeats iteration 2, which it builds on because iteration 2 had the lower script-folded validation WER (16.54 against 16.75).

Iteration 5 (CTC realignment). Rather than discarding misaligned segments, iteration 5 re-cut them. Each training recording was scored with iteration 4's model in 12 s windows every 8 s, chosen after measuring that 30 s windows caused the model to stop emitting partway through (0.928 output characters per transcript character, against 0.971). The recording's transcripts were then aligned in order by a banded CTC Viterbi search within 15 s of their released times. Cuts were placed in the middle of the pauses between transcripts, or within 0.3 s of the transcribed speech where a pause contained speech no transcript accounts for (1.5 h in total, left out). The misalignment filter was then reapplied to the new segments. The median segment start moved by 0.57 s, the filter flagged 1,704 segments instead of 6,645, and 3,032 of the utterances removed in iteration 4 were recovered, giving 48,805 segments (81.09 h). Validation, test and blind keep their released boundaries.

VI. RESULTS AND QUANTITATIVE EVALUATION

All metrics below are the committed results of the training runs described above, measured on the original GPU workstation. They are reported experimental results, not accuracy measurements newly reproduced in the CPU environment of Section VIII, which has no copy of the dataset. Unless stated otherwise, results use KenLM beam search with each iteration's own validation-tuned settings. Each iteration is a single training run with seed 42; no statistical significance testing was performed, and

differences below about one WER point may be within run-to-run variation.

A. Performance across iterations

TABLE IV
WER, T-WER AND CER (%) OF THE FIVE ITERATIONS WITH KENLM BEAM SEARCH

It.	Val WER	Val fold.	Test WER	Test T-WER†	Blind WER	Blind T-WER	Blind CER
1	20.08	17.75	31.29	28.25	21.54	20.22	13.65
2	18.65	16.54	30.19	27.22	21.01	19.74	13.43
3	16.75*	16.75	30.73	27.55	19.36	18.82	11.71
4	18.43	**16.36**	29.98	27.06	20.44	19.20	13.17
5	19.77	17.81	33.04	30.06	19.56	18.15	12.33

Val fold.: script-folded validation WER, Eq. (8), the selection criterion. †Approximate T-WER; test references are not annotated for T-WER. *Measured against iteration 3's own script-normalised references and not comparable with the other rows; its script-folded value is comparable. Bold: selected model. Blind WER and T-WER were confirmed with the organisers' compute_wer.py for every iteration.

Table IV and Fig. 4 report all iterations. Continued training (iteration 2) gave the largest single improvement in script-folded validation WER, from 17.75 to 16.54, and improved every split under both decoders. Script normalisation (iteration 3) reached 16.75, level with iteration 2 within run-to-run variation. The misalignment filter (iteration 4) reached the lowest value, 16.36, and improved on iteration 2 on every split: test WER 30.19 to 29.98, blind WER 21.01 to 20.44 and blind T-WER 19.74 to 19.20. Each of these gains is small, but all point in the same direction. Realignment (iteration 5) was worse on validation (17.81) and test (33.04), and better on blind (19.56 WER, 18.15 T-WER).

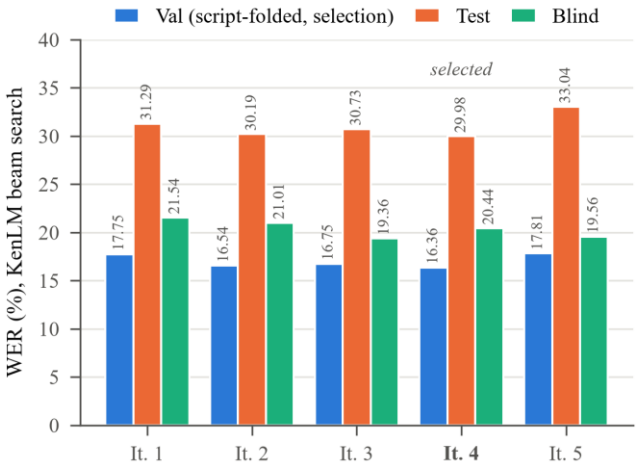


Fig. 4. WER of the five iterations with KenLM beam search. The script-folded validation WER is the selection criterion; test and blind were not used for any decision.

Fig. 5 shows greedy validation WER during training. Iteration 1 converged from 43.9% at update 2,000 to 23.77% at update 37,872. Iteration 4 was below iteration 2 at 21 of 22 checkpoints, by 0.62 points on average, and iteration 5 above iteration 4 at 19 of 22, by 0.66 points on average. Iteration 5's training loss ended far lower than iteration 4's (0.22 against 0.37) while its validation loss ended higher (1.04 against 0.91), consistent with targets that fit the training audio better but differ from the conventions of the validation data.

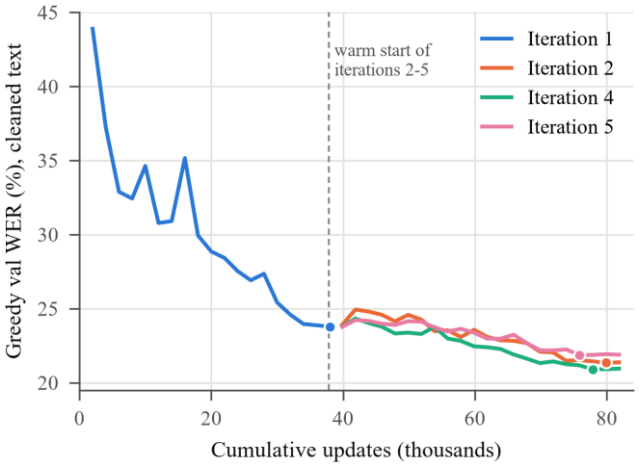


Fig. 5. Greedy validation WER (cleaned text) at each checkpoint. Iterations 2, 4 and 5 start from iteration 1's best checkpoint; dots mark each run's selected checkpoint. Iteration 3 is omitted because it is scored against different (script-normalised) references.

B. Selection of the final model

By Eq. (9), iteration 4 is selected: its script-folded validation WER of 16.36 is the lowest of the five. It is also the best of the five on test WER (29.98). On blind it scores 20.44 WER and 19.20 T-WER, which is not the lowest: iteration 3 has the lowest blind WER (19.36) and iteration 5 the lowest blind T-WER (18.15). Choosing either on that basis would select the model on the test data whose score is then reported, which biases the reported result. Section VII relates both blind gains to properties of the blind references rather than to better recognition on validation data.

C. Generalisation across validation, test and blind sets

Test WER is 9.18–13.48 points higher than blind WER across the iterations, although both are held-out recordings of the same domain. The difference is consistent with poorer segment alignment on test: in the baseline analysis, errors on the first and last word account for 3.95% of test words against 3.35% on blind and 2.22% on validation, and the MUCS organisers observed a similar gap between unaligned and realigned conditions [3]. Test also shares more sentences with training (Table I). Test and blind numbers are therefore not interchangeable, and blind, whose references the organisers report as manually corrected, is the headline split.

Raw and script-folded validation WER differ by about two points for every iteration whose targets keep the original scripts (for example 18.43 against 16.36 for iteration 4). This difference measures how many validation errors are pure script mismatches between Devanagari and Latin spellings of the same English word, which the folded metric forgives. For iteration 3, whose targets are already normalised, the raw and folded values coincide by construction.

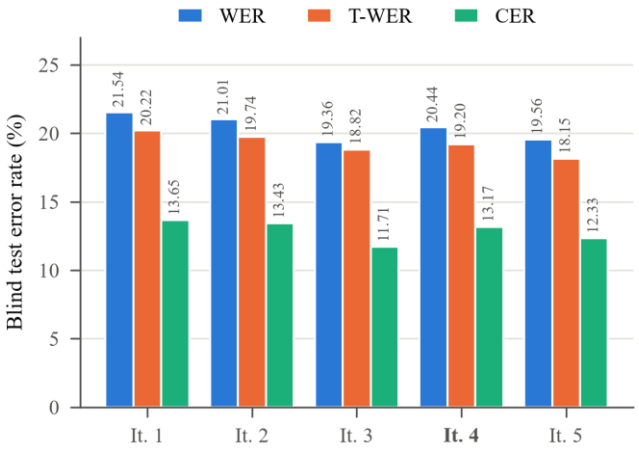


Fig. 6. Blind-test WER, T-WER and CER with KenLM beam search. T-WER forgives Devanagari spellings of English words listed by the organisers; CER is not an official metric.

Fig. 6 compares blind WER, T-WER and CER. T-WER is 0.5–1.4 points below WER for every iteration, and CER varies far less than WER across iterations except where the script of English words changes (iteration 3), because a script change alters every character of a word.

D. Effect of language-model decoding

TABLE V
GREEDY AND KENLM BEAM-SEARCH WER (%) AND THE LM GAIN

It.	Val fold. greedy	Val fold. beam	Gain	Blind greedy	Blind beam	Gain
1	22.11	17.75	4.36	24.96	21.54	3.42
2	19.89	16.54	3.35	23.80	21.01	2.79
3	19.50	16.75	2.75	21.87	19.36	2.51
4	19.40	16.36	3.04	23.34	20.44	2.90
5	20.59	17.81	2.78	22.34	19.56	2.78

Table V isolates the LM. Beam search lowers blind WER by 2.51–3.42 points and script-folded validation WER by 2.75–4.36 points. The gain shrinks as the acoustic model improves: on blind it falls from 3.42 points for iteration 1 to 2.90 for iteration 4. For the baseline, the LM lowered blind CER by only 0.39 points while lowering WER by 3.42, so most of its benefit is in snapping nearly correct spellings onto known words (for example “syzan interpreter” to “python interpreter”). Section VII shows that this comes at the cost of more script errors.

E. Comparison with published systems

TABLE VI
HINDI–ENGLISH BLIND-TEST RESULTS (%)

System	WER	T-WER	Source
Official baseline (GMM-HMM)	25.53	23.80	Published [3]
Chadha et al., wav2vec 2.0 + CTC + LM	21.77	20.75	Published [6]
Dual-script E2E (IITM-SMT-Lab)	22.06	20.97	Leaderboard [20], [21]
CSTR (CNN-TDNN, LF-MMI)	20.41	15.64	Leaderboard [19], [21]
JHU-CLSP/GoVivace (hybrid)	16.43	15.51	Leaderboard [9], [21]
This work, iteration 1 (baseline)	21.54	20.22	Measured
This work, iteration 4 (selected)	**20.44**	**19.20**	Measured

Published values are reported by their authors or the challenge leaderboard and were not reproduced here. This work’s results were obtained after the challenge with the released blind references and the official scoring script; they are not leaderboard submissions, and systems differ in external data and language-model text.

Table VI places the results beside published blind-test numbers. The selected model is 5.09 WER and 4.60 T-WER points below the official baseline and 1.33 WER and 1.55 T-WER points below the published result of the closest method [6], whose 5-gram LM was trained on the external IndicCorp text and decoded with a beam of 3,000. The comparison is indicative only: the encoder checkpoint may differ from theirs, and the result is a single run. Among published Hindi–English leaderboard entries, only the two hybrid systems with extensive lexicon and data engineering report lower WER [9], [19].

VII. ERROR ANALYSIS

This section analyses the selected model on blind with beam search unless stated otherwise (47,764 reference words in 4,026 scored utterances). Words are labelled by script, and a Latin word or a Devanagari word in the organisers’ word list counts as English; transliteration and homophone labels come from phonetic rules and are approximate.

A. Error types

TABLE VII
BLIND-TEST WORD ERRORS, BASELINE AND SELECTED MODEL (BEAM)

Category	It. 1	It. 4	Change
Other substitutions	4,129	3,758	−371
Right word, wrong script	608	572	−36
Homophone spellings	342	333	−9
Incorrect transliteration	279	326	+47
Deletions	3,911	3,817	−94
Insertions	1,019	955	−64
Total (WER %)	10,288 (21.54)	9,761 (20.44)	−527
Error rate, Hindi words	12.01%	11.37%	−0.64
Error rate, English words	31.04%	29.58%	−1.46

Word-group error rates count substitutions and deletions per reference word (31,413 Hindi and 15,301 English words).

Table VII and Fig. 7(a) break down the errors. Deletions (3,817) and substitutions without a phonetic relation to the reference (3,758) dominate. English words, 32% of blind reference words, carry 51.4% of word errors, with an error rate of 29.58% against 11.37% for Hindi words, in line with the MUCS organisers’ observation [3]. Words recognised correctly but written in the wrong script cost 572 errors (1.20 points); the LM adds such errors, because its training text writes English words in both scripts. For the selected model, the right-word-wrong-script count rises from 487 with greedy decoding to 572 with beam search, while other substitutions fall from 5,219 to 3,758.

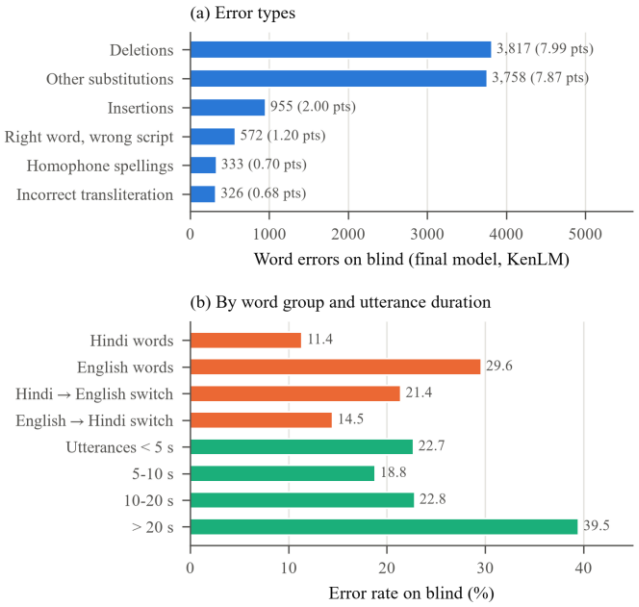


Fig. 7. Errors of the selected model on blind (beam search). (a) Error counts by type, with their contribution in WER points. (b) Error rates for Hindi and English words and for the first word after a language switch (substitutions and deletions per reference word), and WER by utterance duration.

B. Script variation, transliteration and homophones

Table VIII lists verbatim examples from the selected model's blind output. The most frequent confusions are script pairs: "page" written as पेज (108 times), "save" as सेव (62) and "click" as क्लिक (58). Incorrect transliterations include "key" rendered as की, which is also the Hindi postposition "of", and English terms such as "nucleotide" spelled in Devanagari forms absent from the organisers' list. Homophone errors are spelling variants with no acoustic difference, such as गयी/गई and लिये/लिए, which transcribers use inconsistently in every split [5].

TABLE VIII
EXAMPLES FROM THE SELECTED MODEL'S BLIND OUTPUT

Category	Reference excerpt	Hypothesis excerpt
Wrong script	base pairs	base पेयर्स
Wrong script	format में save	format में सेव
Transliteration	enter key दो बार	enter की दो बार
Homophone	दिखाई गयी	दिखाई गई
Rare term	fasta और genbank files	master और genvinc files
Switch point	data files parse करना	data files pass करता है
Edge insertion	tutorial में हम सीखेंगे	इस tutorial में हम सीखेंगे
Edge deletion	स्वागत है इस	स्वागत है
Spoken digits	version 14 . 10	version 1410

C. Language switches, duration and segment boundaries

Errors concentrate at language switches into English: the first word after a Hindi-to-English switch is wrong 21.41% of the time, against 14.47% after an English-to-Hindi switch (Fig. 7(b)). At a switch into English, the model must change phonology and script at once, often on a rare technical term such as "genbank", whereas the word after a switch back into Hindi is typically a frequent function word. By duration, WER is lowest for 5–10 s utterances (18.80%), higher for those under 5 s (22.70%), where boundary effects weigh more, and highest above 20 s (39.48%), where long English command sequences and spelled-out input such as file names lose their word boundaries.

Segment boundaries are a major error source. The most deleted token is the spoken symbol "." (204 times), and the three most inserted words (है, अब, यह) typically begin or end a sentence. In the examples of Table VIII, the edge insertion of इस is a word spoken in this segment's audio but assigned by the transcript to the previous segment. In the baseline analysis, 51% of insertions fell before the first or after the last reference word, and excess edge deletions alone cost an estimated 0.9 blind WER points that no acoustic model can recover with the released segmentation.

D. Why realignment helps blind but not validation

TABLE IX
CHANGE IN GREEDY WER (POINTS) FROM ITERATION 4 TO 5, BY ERROR POSITION

Error position	Val	Test	Blind
Before the first correct word	+0.78	+0.96	−0.20
Between first and last correct word	−0.75	−1.10	+0.36
After the last correct word	+0.82	+1.48	−1.04
No word correct	+0.18	+0.83	−0.13
Total	+1.02	+2.19	−1.00

Table IX counts each greedy-decoding error by its position in the utterance; a segment boundary can only cause errors at the edges. Inside utterances, iteration 5 makes fewer errors than iteration 4 on validation and test. At the edges of validation and test it makes more, chiefly deletions of words that the references include although the cut has partly removed their audio (deletions after the last correct word rise from 402 to 677 on validation). Iteration 4, trained on the same kind of cuts, learned to complete such words; iteration 5, trained on segments that contain exactly their transcript, did not. On blind the same habit becomes an error for iteration 4: it inserts 908 words before the first or after the last correct word, against 274 for iteration 5, because blind references appear to stop where the audio does (one ends on देख, a word cut short). This interpretation is supported by inspected examples but remains a hypothesis about annotation practice.

A further check re-cut the validation set with the same realignment procedure, for analysis only, since the re-cut uses validation references. On the 1,578 validation segments whose released cut contains only its own speech, the two models tie (16.19 against 16.17 greedy WER). On the 738 segments whose released cut passes through speech, iteration 5 is worse (34.76 against 31.63). With all segments re-cut, iteration 5 is better (15.39 against 16.41), even though the cuts were placed with iteration 4's model. Re-cutting lowers both models' greedy WER by about 5 points, an optimistic estimate of what the released boundaries cost.

VIII. IMPLEMENTATION AND REPRODUCIBILITY

A. Software architecture

The system is implemented in Python with PyTorch [31] and Hugging Face Transformers [32]. Modules cover data download with checksum verification and manifest construction, the tokenizer, model loading with blank remapping, training with exact resumption, a logit cache that stores float16 log-posteriors so that both decoders see identical acoustics, LM construction, decoder tuning, evaluation, error analysis and plotting. Training code refuses any split other than training and validation, and the LM builder refuses non-training text. The repository contains 118 unit tests covering the metrics (checked against hand-worked cases and an independent WER implementation), the official scoring rules, the data guards, decoding and the

CPU inference path. Every training run records its configuration, the version-control commit and whether the working tree was clean.

B. Research experiments versus the inference application

The experiments of Sections V–VII were run on an RTX 3050 6 GB laptop GPU with an AMD Ryzen 7 7840HS CPU, Windows 11, Python 3.14, PyTorch 2.14 and pyctcdecode 0.5.0. After the experiments, an inference application was added for CPU-only machines (Fig. 8). It is separate from the research results: it reuses the trained iteration 4 checkpoint and decoder settings but changes no model, setting or reported metric.

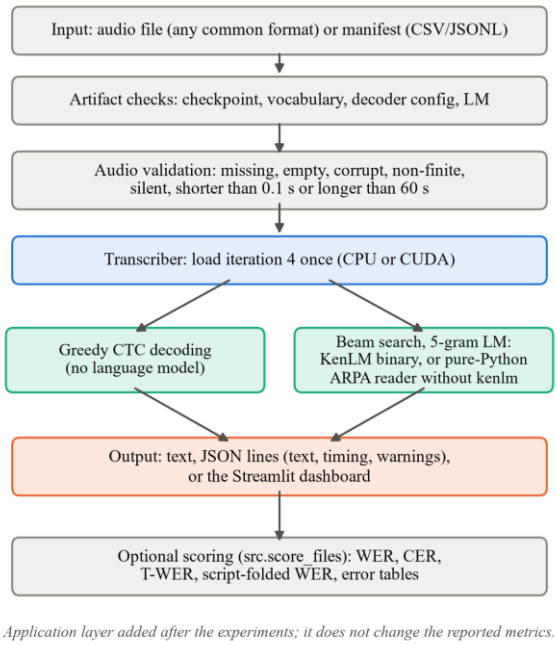


Fig. 8. Inference workflow of the application layer developed after the experiments. It runs on CPU or GPU and reuses the selected model and its validation-tuned decoder settings.

A `Transcriber` class loads the checkpoint and, for beam decoding, the LM once, validates each input (missing, empty, corrupt, non-finite, silent, shorter than 0.1 s or longer than 60 s) before the model runs, and returns text with timing and warnings. Utterances are processed one at a time, because the encoder's group normalisation makes padded batches change the features. Without a GPU, decoding takes a few seconds per clip after a one-time load of 5–40 s. Because the `kenlm` package has no Windows wheel, a pure-Python reader of the same ARPA file was implemented for beam search. Over 2,003 sentences (26,145 words, including shuffled and out-of-vocabulary sentences), its per-word log-probabilities differ from KenLM's by at most 6.3×10^{-7}, which is float32 rounding, and beam decoding with the two backends produced identical text in all 61 test cases. Parsing the ARPA file costs about 3.5 s and 250 MB per load. A scoring tool reproduces the committed iteration 4 metrics exactly from the saved hypotheses, and a Streamlit dashboard provides an overview, CPU transcription of uploaded audio, and the results table. CPU inference was verified end to end on synthetic audio only; no MUCS audio was transcribed on CPU, and no accuracy figure in this paper comes from the CPU path.

C. Reproducibility

The dataset is downloaded from OpenSLR with checksums [4]; model weights and data are not stored in the repository. Each iteration is reproduced by its configuration file, and the evaluation script tunes the decoder on validation data and commits the settings before decoding test and blind. Exact package versions are recorded in a lock file. Every number in this paper is read from committed result files, and the figures are regenerated from them by a script distributed with the paper.

IX. Discussion

Language-model decoding. A 5-gram LM trained on only 50,509 transcripts lowered blind WER by about three points for every iteration, more than any training intervention. Its benefit is lexical: it corrects misspellings onto known words and barely changes CER. It also reproduces the inconsistencies of its training text, writing more English words in Devanagari, and it cannot help with words absent from its 17,005-word vocabulary, which include 8.47% of blind's Latin-script tokens. The shrinking LM gain as the acoustic model improves (Table V) shows the two components correcting overlapping errors; the tuned LM weight and insertion score differ between iterations, so decoder settings must be re-tuned for each acoustic model.

Acoustic versus data interventions. The largest validation gain came from more training, not from a change of data or targets, indicating that the baseline was undertrained under the single-GPU budget. Removing 7% of the training utterances improved every split, consistent with reports that tossing badly aligned segments helps hybrid systems on this corpus [9]. Realignment produced a model that recognises correctly segmented speech better, yet scored worse under the released validation and test segmentation. A data-quality intervention can therefore be beneficial for recognition and harmful for a benchmark score at the same time, depending on how the evaluation data were segmented and transcribed.

Mixed-script evaluation. The disagreement between validation and blind for iterations 3 and 5 shows that evaluation conventions are part of the task definition for code-switched ASR. Blind writes English in Latin script far more often than the training data and appears to transcribe only fully audible words; a model can gain on blind by matching these conventions without recognising better. Script-folded scoring removes the first effect from model selection, and T-WER removes part of it from the official score. No standard metric removes the effect of segment boundaries. Reporting raw WER, T-WER and a script-insensitive WER together, as done here, makes such effects visible.

Generalisation and practical implications. Test WER above 29% for every model shows that results on this corpus depend strongly on segmentation quality, and that a single split can mislead. For deployed Indian multilingual ASR, where audio would typically be segmented at pauses by a voice-activity detector rather than at whole seconds, the re-cut analysis suggests that realigned training could be preferable, but no benchmark result in this study measures that setting. A lightweight self-supervised CTC model with an n-gram LM is competitive with much larger systems on a modest GPU, and it can run on CPU for inference.

X. Limitations and Future Work

- **Single seed.** Every result comes from one training run with seed 42. No variance was measured, so differences below about one WER point, including iteration 4's 0.18-point validation lead over iteration 2, may not be reproducible.
- **Dataset diversity.** All data are spoken programming and science tutorials. Results may not transfer to conversational, noisy or telephone speech, to other domains, or to other speaker populations; no speaker labels exist, so the validation split is not speaker-disjoint.
- **Accent and pronunciation.** No accent or pronunciation analysis was possible, because the corpus provides no speaker metadata; English technical terms pronounced with Hindi phonology remain a dominant error source.
- **Acoustic errors.** Rare English terms, spelled-out commands and long English passages are recognised poorly, and spoken punctuation and digits are frequently wrong.
- **Language-model vocabulary.** The LM is trained on 605 k words of in-domain transcripts only, without the external text used by other systems, and has a closed 17,005-word vocabulary.
- **Evaluation constraints.** T-WER is official only on blind and approximate on test; the script map used for selection depends on training data; segment boundaries of the evaluation sets are imprecise. The encoder's identity with that of Chadha et al. is unverified, and the published comparisons are not controlled.

Future work should repeat the key comparisons with several seeds; evaluate alternative encoders such as XLS-R and Whisper on the same split [2], [12]; adopt transliteration-aware decoding that treats Latin and Devanagari spellings of a word as equivalent; train the LM on larger normalised text; normalise spoken symbols and digits in training targets as the scorer does; and evaluate on audio segmented by voice-activity detection to measure the deployment benefit of realigned training.

XI. Conclusion

This paper presented a controlled study of a wav2vec 2.0 CTC recogniser with 5-gram LM decoding for Hindi–English code-switched speech on MUCS 2021. With every choice made on validation data, the selected model, trained with continued fine-tuning and a misalignment filter, reaches 20.44% WER and 19.20% T-WER on the blind test, 5.09 and 4.60 points below the official baseline, on a single 6 GB GPU. Continued training and removal of misaligned training utterances improved every split, whereas script normalisation and CTC realignment improved the blind test but not the validation set used for selection. The analysis traces these disagreements to differences in how the splits write English words and transcribe speech at segment boundaries, and shows that English technical vocabulary, script inconsistency and segment boundaries account for most remaining errors. These findings indicate that, for code-switched ASR, data quality and evaluation conventions deserve as much attention as model architecture, and that results should be reported across splits and metrics together with the decision protocol behind them.

Declaration of AI Tools and Software

The authors used generative AI tools throughout this project and disclose each use here.

- **Research phase** (literature and dataset surveys, choice of the replication target, implementation brief): ChatGPT (OpenAI), Claude on claude.ai (Anthropic) and Claude Code (Anthropic).
- **Training, evaluation and analysis pipeline** (Stage 1A to iteration 5, with the scripts, configurations, tests and documentation committed by the second author): written and run with Claude Code using the Claude Opus 5.5 model, which also launched the training and evaluation jobs and drafted the analyses.
- **CPU inference application** (CPU backend, ARPA reader, file scoring and dashboard, committed by the first author): developed with ChatGPT and Codex (OpenAI) and Claude Code.
- **This paper**: text, tables, equations, figures and the scripts that build them were drafted with Claude Code (Claude Opus 5.5) from the repository's documents and result files; references were checked against the literature review and by web search.
- **Presentation slides**: prepared by the authors with assistance from ChatGPT.

No AI tool generated a reported result: every number comes from running the code on the MUCS data and is read from the committed result files. The authors directed the work and take full responsibility for the content of this paper.

Software. Python 3.14, PyTorch 2.14.1 with CUDA 13.0, Hugging Face Transformers 5.22.2 and Hub, NumPy, SciPy, soundfile and PyYAML for the model and data; KenLM 0.3.0, compiled from source with MinGW-w64 and Cython, and pyctcdecode 0.5.0 for the language model and beam search; jiwer 4.0.0 for checking the metric code; the organisers' `compute_wer.py` for official scoring; pytest for testing; Streamlit for the dashboard; Google Colab for an early baseline notebook that was later removed unused; matplotlib 3.11.2 for the figures; Node.js with the docx library, Microsoft Word and PyMuPDF for this document; PptxGenJS for the slides; and Git, GitHub and Visual Studio Code throughout.

References

[1] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 12449–12460.

[2] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," in *Proc. Interspeech*, 2021, doi: 10.21437/Interspeech.2021-329.

[3] A. Diwan *et al.*, "MUCS 2021: Multilingual and code-switching ASR challenges for low resource Indian languages," in *Proc. Interspeech*, 2021, doi: 10.21437/Interspeech.2021-1339.

[4] OpenSLR, "SLR104: MUCS 2021 code-switching speech data (Hindi–English, Bengali–English)," [Online]. Available: https://www.openslr.org/104/

[5] B. M. L. Srivastava and S. Sitaram, "Homophone identification and merging for code-switched speech recognition," in *Proc. Interspeech*, 2018, doi: 10.21437/Interspeech.2018-1171.

[6] H. S. Chadha, P. Shah, A. Dhuriya, N. Chhimwal, A. Gupta, and V. Raghavan, "Code switched and code mixed speech recognition for Indic languages," arXiv:2203.16578, 2022.

[7] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376, doi: 10.1145/1143844.1143891.

[8] K. Heafield, "KenLM: Faster and smaller language model queries," in *Proc. Sixth Workshop on Statistical Machine Translation*, Edinburgh, Scotland, 2011, pp. 187–197.

[9] M. Wiesner *et al.*, “Training hybrid models on noisy transliterated transcripts for code-switched speech recognition,” in *Proc. Interspeech*, 2021, doi: 10.21437/Interspeech.2021-2127.

[10] A. Gupta, H. S. Chadha, P. Shah, N. Chhimwal, A. Dhuriya, R. Gaur, and V. Raghavan, “CLSRIL-23: Cross lingual speech representations for Indic languages,” arXiv:2107.07402, 2021.

[11] H. S. Chadha, A. Gupta, P. Shah, N. Chhimwal, A. Dhuriya, R. Gaur, and V. Raghavan, “Vakyansh: ASR toolkit for low resource Indic languages,” arXiv:2203.16512, 2022.

[12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023, pp. 28492–28518.

[13] K. Taneja, S. Guha, P. Jyothi, and B. Abraham, “Exploiting monolingual speech corpora for code-mixed speech recognition,” in *Proc. Interspeech*, 2019, doi: 10.21437/Interspeech.2019-1959.

[14] Y. Sharma, B. Abraham, K. Taneja, and P. Jyothi, “Improving low resource code-switched ASR using augmented code-switched TTS,” in *Proc. Interspeech*, 2020, doi: 10.21437/Interspeech.2020-2402.

[15] B. M. L. Srivastava, B. Abraham, S. Sitaram, R. Mehta, and P. Jyothi, “End-to-end ASR for code-switched Hindi–English speech,” arXiv:1906.09426, 2019.

[16] A. Antony, S. R. Kota, A. Lade, Spoorthy V, and S. G. Koolagudi, “An improved transformer transducer architecture for Hindi–English code switched speech recognition,” in *Proc. Interspeech*, 2022, doi: 10.21437/Interspeech.2022-10763.

[17] K. Dhawan, D. Rekesh, and B. Ginsburg, “Unified model for code-switching speech recognition and language identification based on concatenated tokenizer,” in *Proc. 6th Workshop on Computational Approaches to Linguistic Code-Switching*, 2023, doi: 10.18653/v1/2023.calcs-1.7.

[18] A. Biswas, O. Shevelev, A. Abdaoui, V. Tyagi, and A. Boumadane, “Adapting Whisper for low-resource Hindi–English code-mix speech with on-the-fly augmentation & LLM-synthesised data,” in *Proc. Interspeech*, 2025, doi: 10.21437/Interspeech.2025-447.

[19] O. Klejch, E. Wallington, and P. Bell, “The CSTR system for multilingual and code-switching ASR challenges for low resource Indian languages,” in *Proc. Interspeech*, 2021, doi: 10.21437/Interspeech.2021-1035.

[20] M. G. Kumar *et al.*, “Dual script E2E framework for multilingual and code-switching ASR,” in *Proc. Interspeech*, 2021, doi: 10.21437/Interspeech.2021-978.

[21] MUCS 2021 organisers, “MUCS 2021 challenge leaderboard,” [Online]. Available: https://navana-tech.github.io/MUCS2021/leaderboard.html

[22] T. Verma, A. Shree, and A. Modi, “ASR for low resource and multilingual noisy code-mixed speech,” in *Proc. Interspeech*, 2023, doi: 10.21437/Interspeech.2023-757.

[23] A. Y. Hannun, A. L. Maas, D. Jurafsky, and A. Y. Ng, “First-pass large vocabulary continuous speech recognition using bi-directional recurrent DNNs,” arXiv:1408.2873, 2014.

[24] R. Kneser and H. Ney, “Improved backing-off for M-gram language modeling,” in *Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 1, 1995, pp. 181–184, doi: 10.1109/ICASSP.1995.479394.

[25] S. F. Chen and J. Goodman, “An empirical study of smoothing techniques for language modeling,” *Computer Speech & Language*, vol. 13, no. 4, pp. 359–394, 1999, doi: 10.1006/csla.1999.0128.

[26] S. Garg, T. Parekh, and P. Jyothi, “Dual language models for code switched speech recognition,” in *Proc. Interspeech*, 2018, doi: 10.21437/Interspeech.2018-1343.

[27] Kensho Technologies, “pyctcdecode: A fast and lightweight Python-based CTC beam search decoder for speech recognition,” version 0.5.0. [Online]. Available: https://github.com/kensho-technologies/pyctcdecode

[28] Carnegie Mellon University, “The CMU Pronouncing Dictionary,” [Online]. Available: http://www.speech.cs.cmu.edu/cgi-bin/cmudict

[29] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[30] D. S. Park *et al.*, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Proc. Interspeech*, 2019, pp. 2613–2617, doi: 10.21437/Interspeech.2019-2680.

[31] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.

[32] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45, doi: 10.18653/v1/2020.emnlp-demos.6.